

Python Assignment-20

1: Design a Python application that creates two separate threads named Even and Odd.

- The Even thread should display the first 10 even numbers.
- The Odd thread should display the first 10 odd numbers.
- Both threads should execute independently using the threading module.
- Ensure proper thread creation and execution

Answer :

```
import threading

def Even():
    print("T1 - Even Thread Started")
    for i in range(2,21,2):
        print(i)
    print("T1 - Even Thread Ended")

def Odd():
    print("T2 - Odd Thread Started")
    for i in range(1,21,2):
        print(i)
    print("T2 - Odd Thread Ended")

def main():
    print("Main Threading Started")
    T1 = threading.Thread(target=Even, name="Even")      #thread 1 Created
    T2 = threading.Thread(target=Odd, name="Odd")        #thread 2 Created

    T1.start()      #Starting Thread 1
    T1.join()

    T2.start()      #Starting Thread 2
    T2.join()

    print("Main Thread Ended")

if __name__ == "__main__":
    main()
```

OUTPUT :

```
PS C:\Users\Sumaney\Desktop\GitHub> python -u "c:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-20\A20Q1.py"
Main Threading Started
T1 - Even Thread Started
2
4
6
8
10
12
14
16
18
20
T1 - Even Thread Ended
T2 - Odd Thread Started
1
3
5
7
9
11
13
15
17
19
T2 - Odd Thread Ended
Main Thread Ended
PS C:\Users\Sumaney\Desktop\GitHub>
```

2: Design a Python application that creates two threads named EvenFactor and OddFactor.

- Both threads should accept one integer number as a parameter.
- The EvenFactor thread should:
 - Identify all even factors of the given number.
 - Calculate and display the sum of even factors.
- The OddFactor thread should:
 - Identify all odd factors of the given number.
 - Calculate and display the sum of odd factors.
- After both threads complete execution, the main thread should display the message: "Exit from main"

Answer :

```
import threading

def EvenFactor(iValues):
    print("\n-----Even Thread Start-----")
    EvenNumbers = []
    for iValue in iValues:
        if iValue%2==0:
            EvenNumbers.append(iValue)
    print("Even Numbers : ",EvenNumbers)
    print("Total Even Number count from your list is : ",len(EvenNumbers))
    print("Addition of Even Numbers : ", sum(EvenNumbers))
    print("-----Even Thread Ended-----\n")
```

```

def OddFactor(iValues):
    print("\n-----Odd Thread Start-----")
    OddNumbers = []
    for iValue in iValues:
        if iValue%2!=0:
            OddNumbers.append(iValue)
    print("Odd Numbers : ",OddNumbers)
    print("Total Odd Number count drom your list is : ",len(OddNumbers))
    print("Addition of Odd Numbers : ", sum(OddNumbers))
    print("-----Even Thread Ended-----\n")

def main():
    print("*****{Start Main Function}*****\n")
    Values = []
    print("Enter the Number : ")
    Values = list(map(int,input().split()))
    print("You Entered list of Numbers is : ",Values)

    T1 = threading.Thread(target=EvenFactor, name="EvenFactor",
args=(Values,))
    T2 = threading.Thread(target=OddFactor, name="OddFactor",
args=(Values,))

    T1.start() #Thread1 Started
    T1.join() #Thread1 Wait for Another Thread

    T2.start() #Thread2 Started
    T2.join() #Thread2 Wait for Another Thread

    print("*****{End Main Function}*****")

if __name__ == "__main__":
    main()

```

OUTPUT :

```
GitHub (Workspace)
Code
PS C:\Users\Sumaney\Desktop\GitHub> python -u "c:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-20\A20Q2.py"
*****{Start Main Function}*****

Enter the Number :
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
You Entered list of Numbers is : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

-----Even Thread Start-----

-----Odd Thread Start-----Even Numbers :
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
Total Even Number count from your list is : 10
Addition of Even Numbers : Odd Numbers : 110 [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
Total Odd Number count from your list is :
10-----Even Thread Ended-----

Addition of Odd Numbers : 100
-----Even Thread Ended-----

*****{End Main Function}*****
PS C:\Users\Sumaney\Desktop\GitHub>
```

3: Design a Python application that creates two threads named EvenList and OddList.

- Both threads should accept a list of integers as input.
- The EvenList thread should:
 - Extract all even elements from the list.
 - Calculate and display their sum.
- The OddList thread should:
 - Extract all odd elements from the list.
 - Calculate and display their sum.
- Threads should run concurrently.

Answer :

```
import threading
import queue

#-----Thread Definations-----
def EvenFactor(iValues,QT1):
    print("\n-----Even Thread Start-----")
    EvenNumbers = []
    for iValue in iValues:
        if iValue%2==0:
            EvenNumbers.append(iValue)
    QT1.put(EvenNumbers)
    print("\n-----Even Thread Ended-----\n")
```

```

def OddFactor(iValues,QT2):
    print("\n-----Odd Thread Start-----\n")
    OddNumbers = []
    for iValue in iValues:
        if iValue%2!=0:
            OddNumbers.append(iValue)
    QT2.put(OddNumbers)
    print("-----Odd Thread Ended-----\n")

def main():
    print("*****{Start Main Function}*****\n")
    Values = []
    print("Enter the Number : ")
    Values = list(map(int,input().split()))
    print("You Entered list of Numbers is : ",Values)

#-----Thread Creations-----
QT1 = queue.Queue()
QT2 = queue.Queue()
T1 = threading.Thread(target=EvenFactor, name="EvenFactor",
args=(Values,QT1))
T2 = threading.Thread(target=OddFactor, name="OddFactor",
args=(Values,QT2))

#-----Thread Operations-----
T1.start() #Thread1 Started
T2.start() #Thread2 Started
T1.join() #Thread1 Wait for Another Thread
T2.join() #Thread2 Wait for Another Thread

#-----After Thread end Return Operations-----
REvenNumbers = QT1.get()
ROddNumbers = QT2.get()

print("\n\n-----{Even Results}-----")
print("Even Numbers : ",REvenNumbers)
print("Total Even Number count from your list is :
",len(REvenNumbers))
print("Addition of Even Numbers : ", sum(REvenNumbers))

print("\n\n-----{Odd Results}-----")
print("Odd Numbers : ",ROddNumbers)
print("Total Odd Number count drom your list is : ",len(ROddNumbers))

```

```

print("Addition of Odd Numbers : ", sum(ROddNumbers))

print("\n*****{End Main Function}*****")

if __name__ == "__main__":
    main()

```

OUTPUT :

```

PS C:\Users\Sumaney\Desktop\GitHub> python -u "c:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-20\A20Q3.py"
*****{Start Main Function}*****

Enter the Number :
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
You Entered list of Numbers is : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

-----Even Thread Start-----
-----Odd Thread Start-----
-----Even Thread Ended-----

-----Odd Thread Ended-----

-----{Even Results}-----
Even Numbers : [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
Total Even Number count from your list is : 10
Addition of Even Numbers : 110

-----{Odd Results}-----
Odd Numbers : [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
Total Odd Number count from your list is : 10
Addition of Odd Numbers : 100

*****{End Main Function}*****
PS C:\Users\Sumaney\Desktop\GitHub>

```

4: Design a Python application that creates three threads named Small, Capital, and Digits.

- All threads should accept a **string as input**.
- The **Small** thread should count & display the **number of lowercase characters**.
- The **Capital** thread should count & display the **number of uppercase characters**.
- The **Digits** thread should count and display the **number of numeric digits**.
- Each thread must also display: ◦ **Thread ID** ◦ **Thread Name**

Answer ;

```

#
=====
===
# Author   : codingwithamar@gmail.com
# File     : A20Q4.py
# Path     : Marvellous-Python-Assignments-P229\Assignment-20\A20Q4.py

```

```

# Subject/Question :
# 4: Design a Python application that creates three threads named Small,
Capital, and Digits.
# Description :
# • All threads should accept a string as input.
# • The Small thread should count & display the number of lowercase
characters.
# • The Capital thread should count & display the number of uppercase
characters.
# • The Digits thread should count and display the number of numeric
digits.
# • Each thread must also display: • Thread ID • Thread Name
#
=====
===

import threading
import queue

def Small(iStrings,iQT1):
    print()
    print("-"*15,"Small Thread Started","-"*15)

    print("Thread Name : ",threading.current_thread().name)
    print("Thread ID : ",threading.get_ident())

    Charlist = []
    for char in iStrings:
        if char.islower():
            Charlist.append(char)
    iQT1.put(Charlist)
    print("-"*15,"Small Thread Ended","-"*15)
    print()

def Capital(iStrings,iQT2):
    print()
    print("-"*15,"Capital Thread Started","-"*15)

    print("Thread Name : ",threading.current_thread().name)
    print("Thread ID : ",threading.get_ident())

    Charlist = []
    for char in iStrings:
        if char.isupper():
            Charlist.append(char)
    iQT2.put(Charlist)

```

```

print("-"*15,"Capital Thread Ended","-"*15,)
print()

def Digit(iStrings,iQT3):
    print()
    print("-"*15,"Digit Thread Started","-"*15)

    print("Thread Name : ",threading.current_thread().name)
    print("Thread ID : ",threading.get_ident())

    Charlist = []
    for char in iStrings:
        if char.isdigit():
            Charlist.append(char)
    iQT3.put(Charlist)
    print("-"*15,"Digit Thread Ended","-"*15)
    print()

def Spaces(iStrings,iQT4):
    print()
    print("-"*15,"Space Thread Started","-"*15)

    print("Thread Name : ",threading.current_thread().name)
    print("Thread ID : ",threading.get_ident())

    Charlist = []
    for char in iStrings:
        if char.isspace():
            Charlist.append(char)
    iQT4.put(Charlist)
    print("-"*15,"Space Thread Ended","-"*15)
    print()

def main():
    print("*"*20,"MAIN PROCESS STARTED","*"*20)
    Strings = input("Enter the Lines : ")
    print("You Entered Lines is :\n",Strings)

    QT1 = queue.Queue()
    QT2 = queue.Queue()
    QT3 = queue.Queue()
    QT4 = queue.Queue()

    T1 = threading.Thread(target = Small, name = "Small", args =
(Strings,QT1))

```



```
T2 = threading.Thread(target = Capital, name = "Capital", args =
(Strings,QT2))
T3 = threading.Thread(target = Digit, name = "Digit", args =
(Strings,QT3))
T4 = threading.Thread(target = Spaces, name = "Space", args =
(Strings,QT4))

T1.start()
T1.join()

T2.start()
T2.join()

T3.start()
T3.join()

T4.start()
T4.join()

Rsmall = QT1.get()
RCapital = QT2.get()
RDigit = QT3.get()
RSpace = QT4.get()

print()
print("*"*20,"COUNTS","*"*20)
print("Small letters count from your lines : ",len(Rsmall))
print("Capital letters count from your lines : ",len(RCapital))
print("Digits Count from your lines : ", len(RDigit))
print("Spaces Count from your lines : ",len(RSpace))

print()
print("*"*20,"DISPLAY","*"*20)
print("Small Letters : ", Rsmall)
print("Capital Letters : ", RCapital)
print("Digits : ", RDigit)
print("Spaces : ", RSpace)

print()
print("*"*20,"MAIN PROCESS ENDED","*"*20)

if __name__ == "__main__":
    main()
```

OUTPUT :

```
Enter the Lines : Amar Bharat Bhandare Patil 9
You Entered Lines is :
Amar Bharat Bhandare Patil 9

----- Small Thread Started -----
Thread Name : Small
Thread ID : 29040
----- Small Thread Ended -----

----- Capital Thread Started -----
Thread Name : Capital
Thread ID : 19516
----- Capital Thread Ended -----

----- Digit Thread Started -----
Thread Name : Digit
Thread ID : 15296
----- Digit Thread Ended -----

----- Space Thread Started -----
Thread Name : Space
Thread ID : 9564
----- Space Thread Ended -----

***** COUNTS *****
Small letters count from your lines : 19
Capital letters count from your lines : 4
Digits Count from your lines : 1
Spaces Count from your lines : 4

***** DISPLAY *****
Small Letters : ['m', 'a', 'r', 'h', 'a', 'r', 'a', 't', 'h', 'a', 'n', 'd', 'a', 'r', 'e', 'a', 't', 'i', 'l']
Capital Letters : ['A', 'B', 'B', 'P']
Digits : ['9']
Spaces : [' ', ' ', ' ', ' ', ' ']

***** MAIN PROCESS ENDED *****
```

5: Design a Python application that creates two threads named Thread1 and Thread2.

- Thread1 should display numbers from 1 to 50.
- Thread2 should display numbers from 50 to 1 in reverse order.
- Ensure that:
 - Thread2 starts execution only after Thread1 has completed.
- Use appropriate thread synchronization

Answer:

```
import threading

EventObj = threading.Event()

def Thread1():
    for i in range(1,51,1):
        print(i)
    EventObj.set() #Signal passed when completed

def Thread2():
    EventObj.wait() #wait for Thread1 ending
    for i in range(50,0,-1):
```

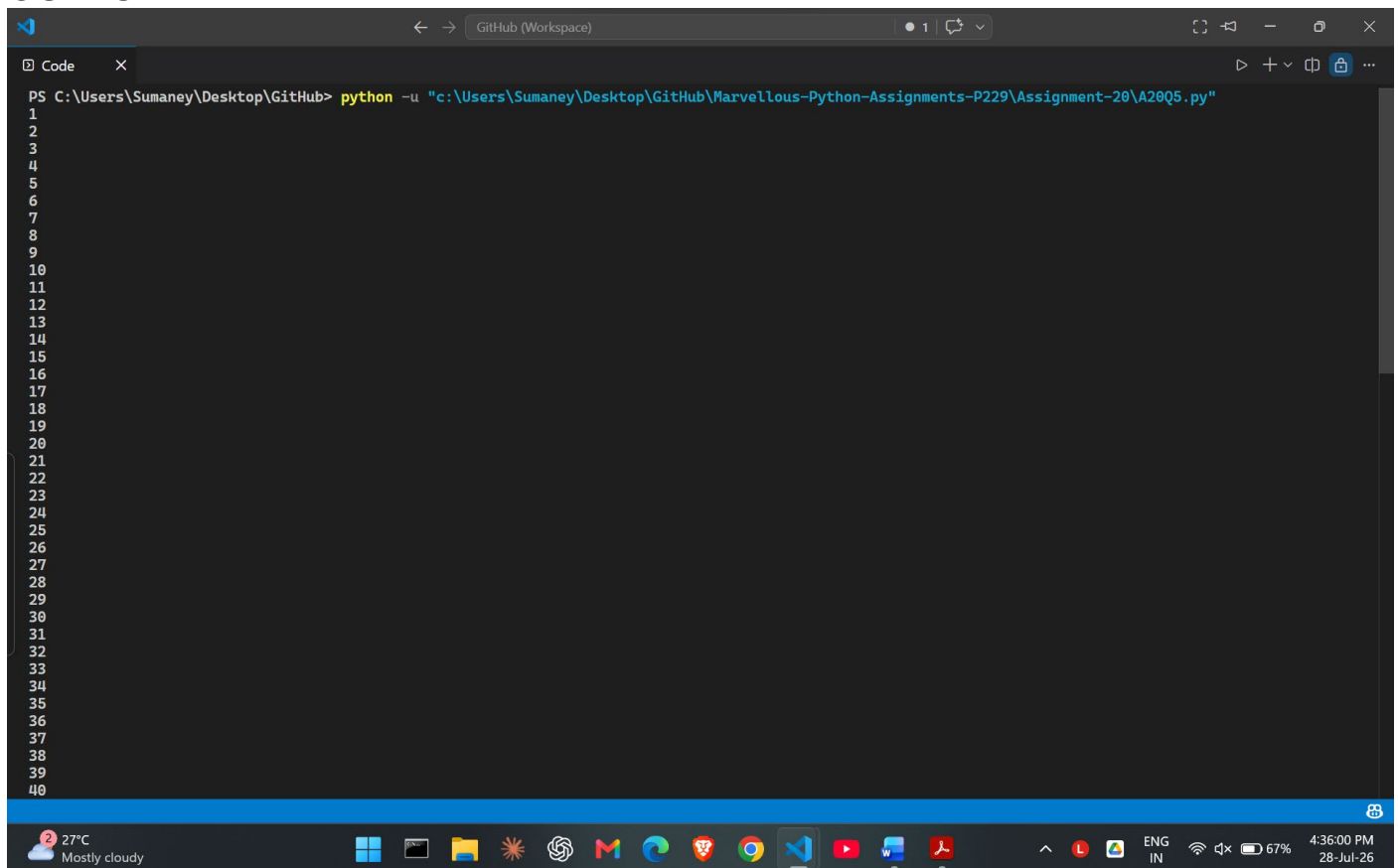
```
        print(i)

def main():
    T1 = threading.Thread(target = Thread1, name ="Thread1")
    T2 = threading.Thread(target = Thread2, name ="Thread2")

    T1.start()
    T2.start()
    T1.join()
    T2.join()

if __name__ == "__main__":
    main()
```

OUTPUT :



```
PS C:\Users\Sumaney\Desktop\GitHub> python -u "c:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-20\A20Q5.py"
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
```

